Universidad de Valladolid

Facultad de Ingeniería Informática

Evaluación y Rendimiento de Sistemas Software

Primera Práctica: Evaluación del rendimiento de un sistema informático con Grafana

Grupo: 14

Autores:
Martín Carmona, Mario
Martínez Vázquez, Elena
Rodrigues Álvarez, Emily Fabiana

Índice

1. Introducción **2**

2. Configuración del entorno **2**
2.1. Instalación y configuración del lenguaje de programación 2
2.2. Configuración de OSHI y Maven 2
2.3. Organización del directorio de trabajo 3
2.4. Preparación de la base de datos 3
2.5. Instalación y configuración de Grafana 4

3. Análisis de los datos y resultados **4**
3.1. Estadísticas de red (Bytes enviados y recibidos) 4
3.2. Carga de CPU por proceso 5
3.3. Carga instantánea de CPU 5
3.4. Conexiones de red activas 6
3.5. Frecuencia instantánea de CPU 7
3.6. Espacio en disco 8
3.7. Operaciones de E/S 9
3.8. Lecturas/escrituras de I/O 9
3.9. Memoria asignada por proceso 9
3.10. Número de procesos en ejecución 9
3.11. Lista detallada de los procesos en ejecución 10
3.12. Uso de memoria RAM y memoria SWAP 11

4. Conclusiones **11**

Índice de figuras

1. 4
2. 5
3. 5
4. 6
5. 7
6. 8
7. 8
8. 9
9. 9
10. 10
11. 10
12. 11
13. 11

1. Introducción

En esta practica, hemos desarrollado un sistema para recopilar y analizar el rendimiento de nuestra maquina virtual. Para ello hemos decidido hacer uso de la biblioteca OSHI, que nos permitirá recopilar medidas sobre aspectos como el procesador, la memoria, entre otros aspectos.

Una vez obtenidos estos datos los almacenaremos en una base de datos, luego los analizaremos en un dashboard de Grafana, que nos permite visualizarlos. Gracias a ello podremos ver estado de nuestra maquina, ademas de poder detectar errores o posibles problemas.

En este informe, se explicara todos los procesos que hemos seguido tanto para la configuración de la maquina virtual como la captura de datos y su almacenamiento en la base de datos.

2. Configuración del entorno

A lo largo de esta práctica, se ha llevado a cabo la configuración de un entorno de desarrollo completo con herramientas esenciales, utilizando una estructura organizada para facilitar el trabajo y evitar conflictos con el sistema operativo principal. A continuación, se detalla cada paso:

2.1. Instalación y configuración del lenguaje de programación

Para trabajar con *Java*, se han seguido los siguientes pasos:

- Se descargó la versión `jdk-11.0.25` y se ubicó en el directorio `/usr/lib/jvm/`, tras extraerlo desde un archivo comprimido almacenado inicialmente en el directorio de trabajo.
- Se añadió la variable de entorno `JAVA_HOME` al archivo `~/.bashrc`, asegurando que el sistema reconozca la ubicación del compilador.
- Se validó la instalación con el comando `java --version`, confirmando que Java se encuentra correctamente configurado.

2.2. Configuración de OSHI y Maven

Se configuró la biblioteca **OSHI** (*Operating System and Hardware Information*) para la recolección de datos del sistema:

- Se clonó el repositorio oficial de OSHI desde GitHub en el directorio de trabajo mediante:

```
git clone https://github.com/oshi/oshi.git
```

- Se instaló `maven` mediante el comando `sudo apt install maven`, herramienta esencial para gestionar las dependencias del proyecto.
- Se ejecutó el siguiente comando para verificar la correcta integración de OSHI:

```
./mvnw test-compile -pl oshi-core exec:java \
    -Dexec.mainClass="oshi.SystemInfoTest" \
    -Dexec.classpathScope="test"
```

  Aunque después de analizarlo como grupo, decidimos que sería menos laborioso generar un archivo `terminal.sh` que, mediante comandos, obtuviese los datos requeridos por la práctica y estos se almacenasen en la base de datos.

2.3. Organización del directorio de trabajo

Todo el desarrollo se realizó dentro del directorio `/home/usuario/erss`, creado específicamente para evitar interferencias con otros archivos del sistema. Dentro de este directorio:

- Se organizó el proyecto en subcarpetas siguiendo buenas prácticas para proyectos Java, facilitando el mantenimiento y la escalabilidad del sistema.
- Se crearon los archivos `metrics.service` y `metrics.timer` para automatizar la ejecución del archivo `terminal.sh`. Los archivos metrics fueron almacenados fuerda del directorio de trabajo, en la ruta `/etc/systemd/system`, y para ejecutarlos se usó el comando `sudo systemctl start metrics.timer`

2.4. Preparación de la base de datos

Además de configurar Java y OSHI, se trabajó con MySQL para almacenar los datos recopilados. Para ello:

- Se accedió y configuró el servidor MySQL utilizando comandos como `sudo mysql -u root -p`.
- Se realizó una prueba de conectividad con la base de datos desde el proyecto, verificando el flujo de datos.

2.5. Instalación y configuración de Grafana

Para visualizar los datos recopilados, se configuró Grafana como sistema de visualización:

- Se instaló y configuró en la máquina virtual.
- Se diseñó un *dashboard* con paneles personalizados, mostrando métricas como uso de CPU, memoria, actividad de disco y estadísticas de red.

3. Análisis de los datos y resultados

En esta parte de nuestra practica, tras realizar los pasos previos explicados anteriormente, realizamos la interpretación de los datos obtenidos, tendencias detectadas y su relevancia.

3.1. Estadísticas de red (Bytes enviados y recibidos)

Se analizaron los bytes enviados y recibidos utilizando gráficos tipo "Time Series". Esto permitió observar tendencias en el tráfico de red:

- Representa los bytes enviados.

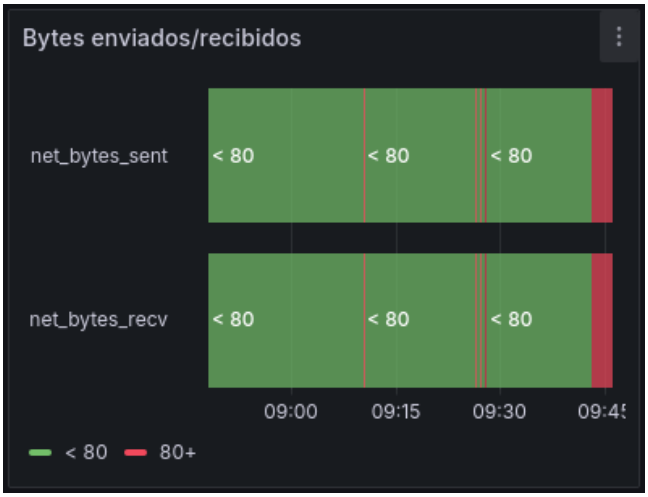


Figura 1

- Muestra la evolución conjunta (líneas separadas) para bytes enviados y recibidos.

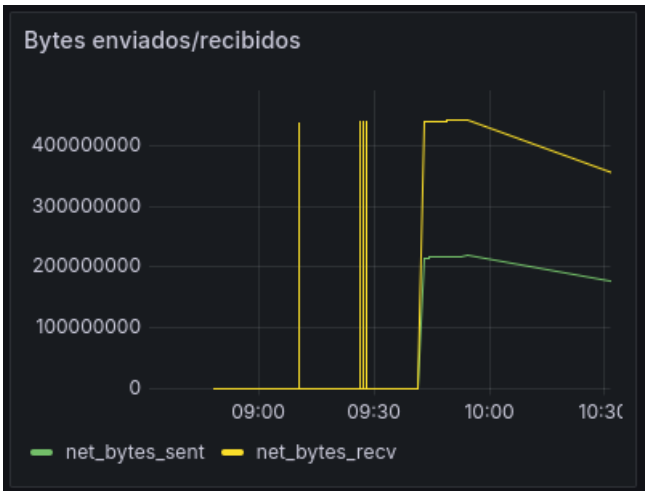


Figura 2

Observamos un tráfico de red fluctuante con picos en momentos de alta actividad del sistema (cuando se realizaron pruebas con el comando `ping`). La cantidad de bytes enviados y recibidos nos permitió detectar posibles cuellos de botella en la red.

3.2. Carga de CPU por proceso

Se visualizó la carga por proceso utilizando gráficos detallados que muestran cómo cada proceso afecta al rendimiento general del sistema, siendo el que más consume y demanda grafana, el visualizador de los datos del sistema.

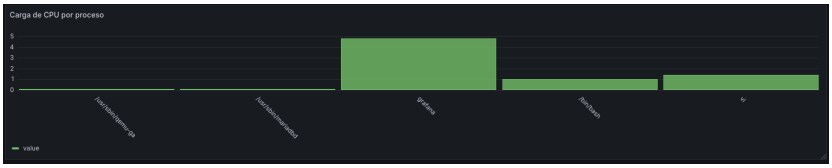


Figura 3

3.3. Carga instantánea de CPU

Usando la gráfica Time series, podemos ver la evolución de la carga a lo largo del tiempo y podemos detectar y analizar los distintos picos o tendencias por las que pasa.

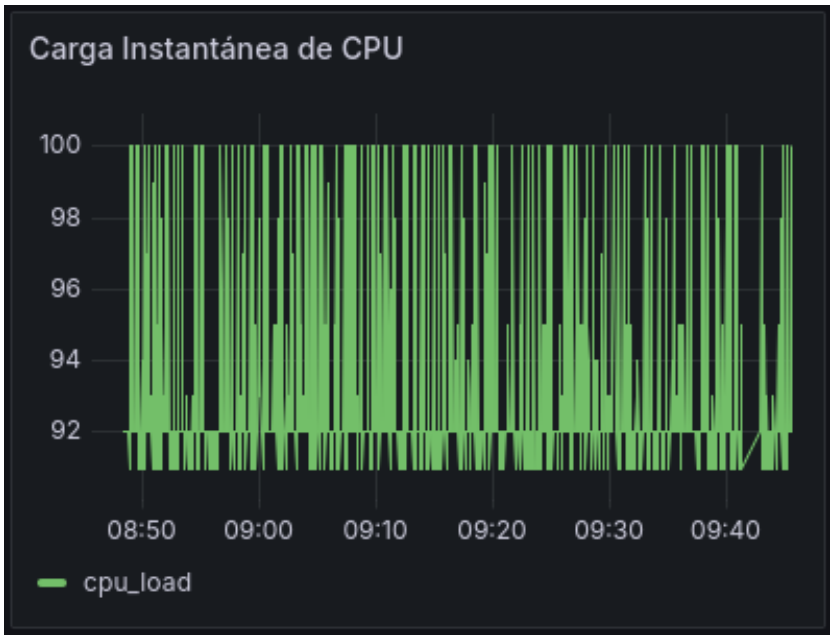


Figura 4

3.4. Conexiones de red activas

Como ha habido actividad de conexión, para visualizar estos datos hemos usando la gráfica "Time Series", para poder ver cómo este número varía, especialmente en picos de tráfico o durante períodos críticos.

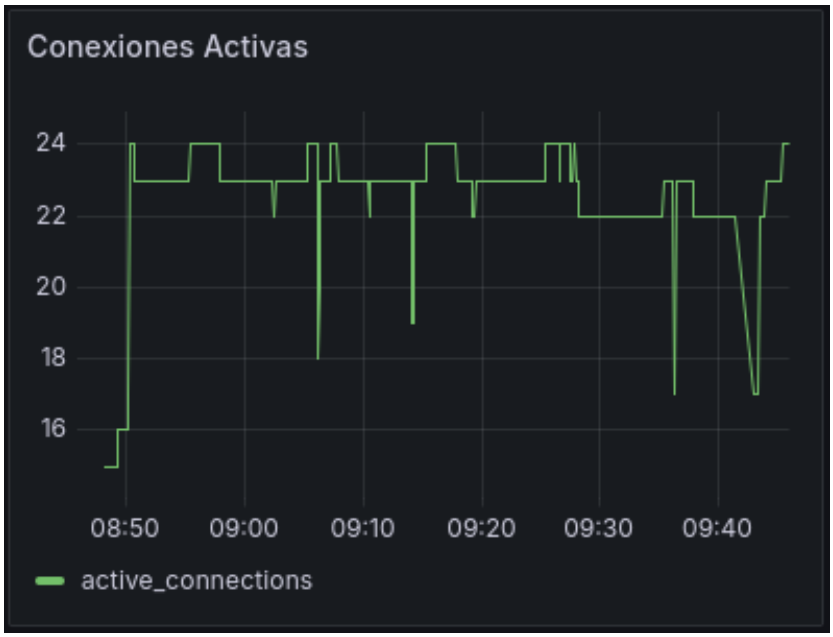


Figura 5

3.5. Frecuencia instantánea de CPU

Esta gráfica muestra la evolución de la frecuencia de la CPU a través del tiempo, como se puede observar es constante debido a que con las configuraciones de energía, de la BIOS, el gobernador de CPU del sistema y las opciones de virtualización junto con el poco uso del procesador es normal que esta permanezca constante.



Figura 6

3.6. Espacio en disco

Monitorizar la evolución del uso del disco (por ejemplo, en sistemas donde el espacio se consume de forma progresiva).



Figura 7

3.7. Operaciones de E/S

Las operaciones de E/S fueron representadas mediante gráficos combinados que muestran lecturas y escrituras separadas, ayudando a identificar picos o caídas en la actividad.

3.8. Lecturas/escrituras de I/O



Figura 8

3.9. Memoria asignada por proceso

Se representó gráficamente la cantidad de memoria asignada a cada proceso activo en el sistema.

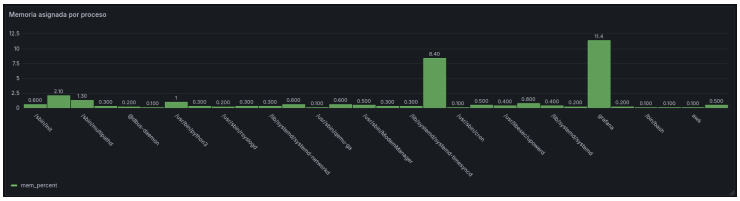


Figura 9

3.10. Número de procesos en ejecución

Usamos el gráfico Time series para analizar la evolución en el tiempo, identificando posibles variaciones repentinas que puedan indicar picos en la carga

del sistema.



Figura 10

3.11. Lista detallada de los procesos en ejecución

El uso de la gráfica "Time Series.[es] la opción ideal para mostrar datos tabulares. Puedes incluir columnas como PID, nombre del proceso, porcentaje de CPU y de memoria, etc. Esta visualización te permitirá ordenar, filtrar y hasta resaltar procesos si es necesario.

Lista detallada de los procesos en ejecución

time	Process ID	Process Name	CPU Usage (%)	Memory Usage (%)
2025-03-30 14:39:21.000	488	/usr/sbin/qemu-ga	0.100	0.100
2025-03-30 14:39:21.000	782	/usr/sbin/mariadbd	0.200	11.2
2025-03-30 14:39:21.000	1166728	grafana	4.40	12.1
2025-03-30 14:02:50.000	1769307	/bin/bash	2	0.100
2025-03-30 14:02:55.000	1770408	vi	2	0.500
2025-03-30 14:10:36.000	1773141	/usr/libexec/fwupd/fwupd	13.3	1.50
2025-03-30 14:04:12.000	1784387	/bin/bash	1	0.100
2025-03-30 14:04:25.000	1786536	/bin/bash	1	0.100
2025-03-30 14:04:28.000	1787073	/bin/bash	1	0.100

Figura 11

3.12. Uso de memoria RAM y memoria SWAP

En esta sección la gráfica "Time Series"nos permite ver la evolución del uso de RAM y SWAP en función del tiempo. Es ideal para detectar cuellos de botella o cambios abruptos.

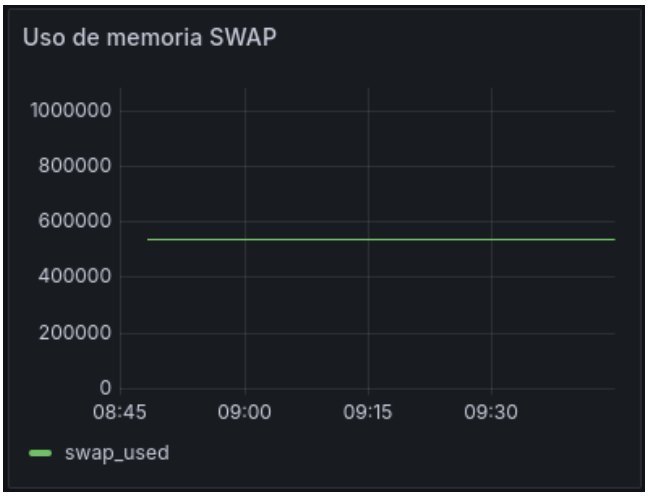


Figura 12

El uso de la memoria SWAP no varía debido a que el SO solo la utiliza en momentos de alta demanda de RAM y según las configuraciones del sistema. Como consecuencia de su poca utilización podemos observar como la gráfica se permanece constante.



Figura 13

4. Conclusiones

El análisis detallado de los datos obtenidos revela que el sistema monitorizado presenta, en líneas generales, un rendimiento aceptable y estable. No obstante, se han identificado algunos aspectos que podrían optimizarse para mejorar la eficiencia global:

Optimización de procesos críticos: Es importante realizar un seguimiento continuo de aquellos procesos que generan elevados picos en la carga de CPU y actividad en la red. Una gestión más eficiente y, de ser necesario, la redistribución de recursos, contribuirá a reducir la sobrecarga en momentos críticos.

Mejora en la gestión de operaciones de E/S: Los picos detectados en las operaciones de lectura y escritura sugieren revisar la planificación de las tareas asociadas al acceso intensivo al disco. Se recomienda evaluar posibles actualizaciones de hardware o ajustes en la configuración para mantener un flujo de datos constante.

Escalabilidad y seguimiento continuo: Implementar un sistema de monitoreo continuo, con comparativas periódicas de las métricas obtenidas, facilitará la detección temprana de anomalías y la planificación de futuras mejoras en la infraestructura del sistema.

Optimización de la distribución de memoria: Aunque el uso de SWAP se mantiene en niveles reducidos, es fundamental seguir controlando la asignación de memoria entre procesos para prevenir cuellos de botella y asegurar que los procesos críticos dispongan de los recursos necesarios.

En definitiva, la práctica ha permitido identificar tanto puntos fuertes como áreas de mejora en el rendimiento del sistema. La aplicación de las recomendaciones aquí planteadas y la monitorización continuada mediante Grafana serán clave para afrontar con éxito futuros desafíos y garantizar un funcionamiento óptimo del sistema.

Referencias

[1] OSHI Developers. *OSHI: Operating System & Hardware Information*. Último acceso: 6 de marzo de 2025. 2025. URL: `https://github.com/oshi/oshi`.